

Lecture 15

Verilog Operators

Contents of the lecture

Verilog Operators

- (1) Bitwise operators
- (2) Logical Operators
- (3) Reduction Operators
- (4) Arithmetic Operator
- (5) Relational Operator
- (6) Equality Operator
- (7) Shift Operator
- (8) Concatenate Operator
- (9) Conditional Operator
- (10) Replication Operator



Dr. Yogesh Mishra
Professor of ECE< GMRIT

(1) Bitwise Operators: Bitwise operators operate on individual bits of operands

Operator type	Operator symbols	Operation performed	Number of operands
Bitwise	~	1's complement	1
	&	Bitwise AND	2
		Bitwise OR	2
	^	Bitwise XOR	2
	~ ^ or ^ ~	Bitwise XNOR	2

Let $A[2:0]$, $B[2:0]$ and $C[2:0]$ are three-bit vectors and f and w are two scalars

(i) The $\sim$ operator forms the 1's complement of the operand.

The statement $C = \sim A$;

produces the result $C(2) = A(2)'$, $C(1) = A(1)'$, and $C(0) = A(0)'$

(ii) The & operator forms bitwise 'AND' operation.

The statement $C = A \& B$;

produces the result $C(2) = A(2).B(2)$, $C(1) = A(1).B(1)$, and $C(0) = A(0).B(0)$

(iii) The | operator forms bitwise 'OR' operation.

The statement $C = A | B$;

produces the result $C(2) = A(2)|B(2)$, $C(1) = A(1)|B(1)$, and $C(0) = A(0)|B(0)$

(iv) The $\wedge$ operator forms bitwise 'XOR' operation.

The statement $C = A \wedge B$;

produces the result $C(2) = A(2) \wedge B(2)$, $C(1) = A(1) \wedge B(1)$, and $C(0) = A(0) \wedge B(0)$

(v) The $\sim \wedge$ or $\wedge \sim$ operator forms bitwise 'XNOR' operation.

The statement $C = A \sim \wedge B$;

produces the result $C(2) = A(2) \sim \wedge B(2)$, $C(1) = A(1) \sim \wedge B(1)$, and $C(0) = A(0) \sim \wedge B(0)$

$\&$	0	1	x
0	0	0	0
1	0	1	x
x	0	x	x

$ $	0	1	x
0	0	1	x
1	1	1	1
x	x	1	x

$\wedge$	0	1	x
0	0	1	x
1	1	0	x
x	x	x	x

$\sim \wedge$	0	1	x
0	1	0	x
1	0	1	x
x	x	x	x

Truth tables for bitwise operators.

Lecture 16

Verilog Operators (Continue)

Contents of the lecture

Verilog Operators

- (1) Bitwise operators
- (2) Logical Operators
- (3) Reduction Operators
- (4) Arithmetic Operator
- (5) Relational Operator
- (6) Equality Operator
- (7) Shift Operator
- (8) Concatenate Operator
- (9) Conditional Operator
- (10) Replication Operator



Dr. Yogesh Mishra
Professor of ECE< GMARIT

(2) Logical Operators: Logical operators operate on individual bits of operands

Operator type	Operator symbols	Operation performed	Number of operands
Logical	!	NOT	1
	&&	AND	2
		OR	2

Let $A[2:0]$, $B[2:0]$ and $C[2:0]$ are three-bit vectors and f and w are two scalars

(i) The $!$ operator forms the 1's complement of the operand.

(a) The $!$ operator has the same effect on a scalar operand as the $\sim$ operator.

For Example

$f = !w$ will produce $f = w'$

$f = \sim w$ will produce $f = w'$

(b) The $!$ operator has the different effect on a vector operand

For Example

The statement $f = !A;$

produces the result $f = [A(2)+A(1)+A(0)]'$

(ii) The && operator implement 'AND' operation

The statement $f = A \&\& B$;
produces the result $f = [A(2)+A(1)+A(0)] \cdot [B(2)+B(1)+B(0)]$

(iii) The || operator implement 'OR' operation

The statement $f = A \parallel B$;
produces the result $f = [A(2)+A(1)+A(0)] + [B(2)+B(1)+B(0)]$

(3) Reduction Operators: The reduction operators perform an operation on the bits of a single vector operand and produce a one-bit result.

Operator type	Operator symbols	Operation performed	Number of operands
Reduction	&	Reduction AND	1
	~&	Reduction NAND	1
		Reduction OR	1
	~	Reduction NOR	1
	^	Reduction XOR	1
	~ ^ or ^ ~	Reduction XNOR	1

(i) The & operator implement 'AND' operation on all bits of single vector operand and produce single bit result

The statement `f=& A;`
produces the result `f = [A(2).A(1).A(0)]`

(ii) The ~& operator implement 'NAND' operation on all bits of single vector operand and produce single bit result

The statement `f=~& A;`
produces the result `f = [A(2).A(1).A(0)]'`

(iii) The `|` operator implement 'OR' operation on all bits of single vector operand and produce single bit result

The statement `f = | A;`
produces the result `f = [A(2)+A(1)+A(0)]`

(iv) The `~|` operator implement 'NOR' operation on all bits of single vector operand and produce single bit result

The statement `f = ~| A;`
produces the result `f = [A(2)+A(1)+A(0)]'`

(v) The $\wedge$ operator implement 'XOR' operation on all bits of single vector operand and produce single bit result

The statement $f = \wedge A;$
produces the result $f = [A(2) \text{ xor } A(1) \text{ xor } A(0)]$

(vi) The $\sim\wedge$ operator implement 'XNOR' operation on all bits of single vector operand and produce single bit result

The statement $f = \sim\wedge A;$
produces the result $f = [A(2) \text{ xor } A(1) \text{ xor } A(0)]'$

Lecture 17

Verilog Operators (Continue)

Contents of the lecture

Verilog Operators

- (1) Bitwise operators
- (2) Logical Operators
- (3) Reduction Operators
- (4) Arithmetic Operator
- (5) Relational Operator
- (6) Equality Operator
- (7) Shift Operator
- (8) Concatenate Operator
- (9) Conditional Operator
- (10) Replication Operator



Dr. Yogesh Mishra
Professor of ECE< GMARIT

(4) Arithmetic Operators: They perform standard arithmetic operations.

Operator type	Operator symbols	Operation performed	Number of operands
Arithmetic	+	Addition	2
	-	Subtraction	2
	-	2's complement	1
	*	Multiplication	2
	/	Division	2

(i) $C=A+B$; puts the three-bit sum of A plus B into C

(ii) $C=A-B$; puts the difference of A and B into C

(iii) $C=-A$; places the 2's complement of A into C .

Example of “+” operator

```
module adder(a,b,c);  
input [2:0]a;  
input [2:0]b;  
output [2:0]c;  
assign c=a + b;  
endmodule
```

(5) Relational Operators: They perform standard relation operations.

Operator type	Operator symbols	Operation performed	Number of operands
Relational	>	Greater than	2
	<	Less than	2
	>=	Greater than or equal to	2
	<=	Less than or equal to	2

The relational operators are typically used as conditions in **if-else** and **for** statements.

An expression that uses the relational operators returns the value 1 if it is evaluated as true, and the value 0 if evaluated as false.

(6) Equality Operators

Operator type	Operator symbols	Operation performed	Number of operands
Equality	==	Logical Equality	2
	!=	Logical Inequality	2

The expression $(A == B)$ is evaluated as true if A is equal to B and false otherwise.

The $!=$ operator has the opposite effect.

(7) Shift Operators

Operator type	Operator symbols	Operation performed	Number of operands
Shift	>>	Right shift	2
	<<	Left shift	2

A vector operand can be shifted to the right or left by a number of bits specified as a constant. When bits are shifted, the vacant bit positions are filled with 0s.

Example1

$B = A \ll 1;$
 results in $B[2] = A[1]$, $B[1] = A[0]$ and $B[0] = 0$

Example2

$B = A \gg 1;$
 results in $B[2] = 0$, $B[1] = A[2]$ and $B[0] = A[1]$

Shift Operators

Example 1 (Right Shift)

$B = A \gg 1;$
results in $B[2] = 0$, $B[1] = A[2]$ and $B[0] = A[1]$

Example 2 (Left Shift)

$B = A \ll 1;$
results in $B[2] = A[1]$, $B[1] = A[0]$ and $B[0] = 0$

Lecture 18

Verilog Operators (Continue)

Contents of the lecture

Verilog Operators

- (1) Bitwise operators
- (2) Logical Operators
- (3) Reduction Operators
- (4) Arithmetic Operator
- (5) Relational Operator
- (6) Equality Operator
- (7) Shift Operator
- (8) Concatenate Operator
- (9) Conditional Operator
- (10) Replication Operator



Dr. Yogesh Mishra
Professor of ECE< GMARIT

(8) Concatenate Operator: This operator concatenates two or more vectors to create a larger vector

Operator type	Operator symbols	Operation performed	Number of operands
Concatenate	{ -, - }	Concatenation	Any number

Concatenate Operator

Example1

Let A and B are two 3 bit vectors and D is a 6 bit vector

Then $D = \{A, B\}$

results six bit vector $D = A[2]A[1]A[0]B[2]B[1]B[0]$

Example2

$E = \{3'b111, A, 2'b00\};$

results 8 bit vector $E = 111A[2]A[1]A[0]00$

(9) Conditional: This operator assigns one of two values depending on a conditional expression

Operator type	Operator symbols	Operation performed	Number of operands
Conditional	?:	Concatenation	3

Syntax

`conditional_expression ? true_expression : false_expression`

If the conditional expression evaluates to 1 (true), then the value of true_expression is chosen; otherwise, the value of false_expression is chosen

Conditional Operator

syntax

`conditional_expression ? true_expression : false_expression`

Example-1

`A = (B < C) ? (D + 5) : (D + 2);`

if B is less than C , the value of A will be $D + 5$
else A will have the value $D + 2$.

The conditional operator can be used both in continuous assignment statements and in procedural statements inside an **always** block

Example: A 2-to-1 multiplexer specified using the conditional operator.

```
module mux2to1 (w0, w1, s, f);
```

```
input w0, w1, s;
```

```
output f;
```

```
assign f = s ? w1 : w0;
```

```
endmodule
```

(10) Replication Operator: This operator allows repetitive concatenation of the same vector, which is replicated the number of times indicated in the replication constant.

Operator type	Operator symbols	Operation performed	Number of operands
Replication	{{}}	Replication	Any number

Replication Operator

Example-1

$\{3\{A\}\}$

results $\{A, A, A\}$

Example-2

$\{4\{2'b10\}\}$

Results the eight-bit vector 10101010

Example-3

$\{2\{A\}, 3\{B\}\}$

Results to $\{A, A, B, B, B\}$